

TradeDoc AI Project Audit Report

Code Quality, Backend Health, Deployment Readiness, and Next-Step Plan

Generated for project folder: Latest_Virtusa_2.0 — Audit date: 15 May 2026

1 Executive Summary

The project is a working full-stack prototype for AI-assisted derivatives confirmation generation. The core flows are present: Next.js frontend, Flask backend, Gemini-based agent pipeline, LaTeX PDF generation, Adobe PDF-to-Word integration hooks, and MongoDB-backed user/document storage.

Overall readiness: Not production/deployment ready yet, but it is close to demo/UAT readiness. The biggest blockers are authentication hardening, frontend lint failures, production WSGI setup, chatbot persistence in MongoDB, and deployment/security cleanup.

Area	Status	Reason
Backend API	Good prototype	Routes are structured, MongoDB connectivity works, document CRUD works, AI extraction works. Needs WSGI server and better error/security handling.
Frontend UI	Functional but needs cleanup	Production build passes, but ESLint fails with 24 errors and 38 warnings.
MongoDB	Working	Signup, document save/list, and readiness checks passed against configured MongoDB. Chatbot is not persisted in MongoDB yet.
Auth	Partially implemented	Signup/login/me exist, but password change is broken, tokens are stored in localStorage, and several security controls are missing.
PDF generation	Working	Authenticated FX NDF PDF generation smoke test passed and returned a valid PDF. Other doc types should be regression tested similarly.
AI agents	Working smoke test	Gemini AI extraction passed on test cases/sample6.txt; classified as FX NDF and populated 18 fields.
Deployment	Not ready	Docker works conceptually, but image is heavy, Flask dev server is used, secrets/runtime configuration need cleanup, and CI/test gates are missing.

2 Project Structure Analysis

2.1 Root folder

- `server.py`: Main Flask backend. Serves static UI, auth APIs, document CRUD, AI extraction, PDF generation, Word conversion, validation, and health routes.
- `agents/`: Lightweight LangGraph-style agent pipeline: classifier, extractor, PDF compiler, validator, Gemini helper, state graph runner.
- `templates/`: LaTeX/Jinja2 PDF generators for FX NDF, IRS, CDS, and Equity TRS.

- `ui-app/`: Next.js 16 + React 19 frontend.
- `test_cases/`: Email/sample inputs for extraction testing.
- `output_confirmations/`: Generated PDFs/TEX/DOCX. Should remain runtime/generated data, not source code.
- `Dockerfile`, `docker-compose.yml`, `entrypoint.sh`: Container deployment setup.

2.2 Folders that should not be committed/deployed as source

- `.venv/`, `ui-app/node_modules/`, `ui-app/.next/`, `ui-app/out/`, `__pycache__`, `*.pyc`, `.DS_Store`, generated PDF output.
- Your `.gitignore` already covers most of these. Good hygiene is present, but the working folder currently contains generated/vendor artifacts.

3 What I Tested

Check	Result	Notes
Python syntax compile	PASS	<code>server.py</code> , <code>agents</code> , and template generator files compiled successfully.
Next.js production build	PASS	<code>npm run build</code> completed successfully. Static pages generated for <code>/</code> , <code>/dashboard</code> , <code>/login</code> , <code>/signup</code> .
ESLint	FAIL	62 total issues: 24 errors, 38 warnings. This should block production release.
Flask startup	PASS	Server starts and reports MongoDB connected successfully.
Health route via Flask test client	PASS	<code>/health/ready</code> returned <code>ready</code> with MongoDB <code>true</code> and Gemini configured <code>true</code> .
Auth signup smoke test	PASS	Test user signup returned 201 and token.
Document save/list smoke test	PASS	Draft document saved to MongoDB and listed successfully.
Password change smoke test	FAIL	Endpoint crashes with <code>KeyError: 'password'</code> because code reads <code>user['password']</code> instead of <code>password_hash</code> .
FX NDF PDF generation	PASS	Authenticated generation returned HTTP 200, <code>application/pdf</code> , and a job-scoped file id.
Gemini AI extraction	PASS	<code>sample6.txt</code> classified as FX NDF and extracted 18 populated fields.

4 Backend Health Review

4.1 Strengths

- Backend has clear route coverage: auth, profile, document CRUD, AI extraction, PDF generation, Word conversion, validation, liveness/readiness.
- MongoDB connection is lazy-loaded and uses indexes for users, documents, and PDF jobs.

- Atlas support is thoughtful: TLS options and certifi handling are present.
- Generated PDFs are job-scoped under user directories, which is much better than a shared flat temp folder.
- AI endpoints validate missing API key and input length for extraction.
- LaTeX generation uses argument-list subprocess calls, not shell execution, reducing command injection risk.

4.2 Issues and risks

- **Password change bug:** `/api/me/change-password` checks `user['password']` and writes `password`; `signup/login` use `password_hash`. This endpoint currently fails.
- **Chat endpoint is unauthenticated:** `/api/chat` does not use `@require_auth`. Any user can call it if they know the route.
- **Flask development server:** The app currently runs via `app.run`. Production should use Gunicorn/uWSGI behind a reverse proxy or platform runtime.
- **Auth token model:** Its dangerous signed tokens are okay for a prototype, but production should use secure HTTP-only cookies or a stronger JWT/session design with revocation strategy.
- **No rate limiting:** AI, login, signup, validation, and PDF generation endpoints should be rate limited.
- **No CSRF strategy:** If switching to cookies, add CSRF protection. With bearer tokens, secure storage is still an issue.
- **Input validation is uneven:** JSON schemas exist on frontend, but backend does not deeply validate document payloads before generating PDFs.
- **Global single-file backend:** `server.py` is large. It works, but maintainability will improve by splitting routes/services.

5 Frontend Quality Review

5.1 Strengths

- UI is feature-rich: dashboard, AI extraction, manual forms, PDF preview, settings, saved documents, and chat copilot.
- Production build passes, which means TypeScript compilation and static export are not blocking right now.
- API helper centralizes base URL and auth headers.
- Dashboard flow integrates document save, generate PDF, convert to Word, validate, and load saved documents.

5.2 Key frontend gaps

- ESLint fails. Main categories: `no-explicit-any`, unused variables/imports, unescaped apostrophe, React hooks lint around `setState` in effects, and image optimization warnings.
- Auth state is stored in `localStorage`. This is easy but not ideal for production because XSS can read tokens.
- Signup terms checkbox is visual only; it does not block submission.

- Forgot password links are placeholders.
- Chat history is stored in browser `localStorage`; it is not connected to MongoDB.
- Some dashboard state and type logic is large and could be split into hooks/services.

6 Authentication Status

You mentioned auth is still pending. My finding: **auth is partly built, but not production complete.**

Existing pieces:

- Signup: implemented and tested successfully.
- Login: implemented.
- Current user: `/api/auth/me` implemented.
- Profile update: implemented.
- Route protection: most document/PDF/AI routes are protected with bearer token auth.

Missing/fix-needed pieces:

- Fix password change: use `password_hash` consistently.
- Protect `/api/chat` with `@require_auth`.
- Add logout/session expiration UX.
- Add forgot password/reset password flow.
- Add email verification if this will be public.
- Add rate limiting and login abuse protection.
- Move tokens away from `localStorage` for production, preferably HTTP-only secure cookies.
- Add backend validation for profile fields and password policy.

7 Chatbot + MongoDB Integration Plan

Current state:

- Frontend stores chat history in `localStorage`.
- Frontend sends the history array to `/api/chat` with every message.
- Backend uses Gemini and returns reply/action.
- Backend does not persist conversations or messages in MongoDB.

Recommended MongoDB design:

- `chat_sessions`: `_id`, `user_id`, `title`, `created_at`, `updated_at`, `last_message`, `metadata`.
- `chat_messages`: `_id`, `session_id`, `user_id`, `role`, `content`, `action`, `model`, `created_at`.
- Indexes: `(user_id, updated_at)`, `(session_id, created_at)`.

Recommended API endpoints:

- POST `/api/chat/sessions`: create session.
- GET `/api/chat/sessions`: list sessions for user.

- GET `/api/chat/sessions/<id>/messages`: load messages.
- POST `/api/chat`: accept `session_id` + message, persist user message, call Gemini, persist assistant reply.
- DELETE `/api/chat/sessions/<id>`: delete a session.

8 Deployment Readiness

8.1 Current deployment assets

- Dockerfile installs Python, Node, TinyTeX, MongoDB, app dependencies, builds UI, and runs Flask.
- Compose file provides a separate MongoDB service plus app service.
- Health endpoints exist: `/health/live` and `/health/ready`.

8.2 Deployment blockers

- Use a production WSGI server instead of Flask dev server. Example: `gunicorn -w 2 -b 0.0.0.0:5055 server:app`.
- Dockerfile installs MongoDB inside the app image, while docker-compose also runs MongoDB separately. Choose one model. Recommended: keep MongoDB as a separate service and remove MongoDB from app image.
- Pin dependency versions for repeatable builds. Current requirements use broad `>=` ranges.
- Add CI gates: Python syntax/tests, frontend lint, frontend build, Docker build.
- Add structured logging and avoid printing sensitive runtime details.
- Decide whether generated PDFs are temporary or persisted. The code persists pdf job metadata and stores generated files under runtime volume.
- Add cleanup policy for temp PDFs and old job folders.
- Review `.dockerignore`: comments say `.env` is intentionally included, but Dockerfile does not copy it. Prefer never baking `.env` into image; pass secrets at runtime only.

9 Security Review

Priority fixes before public deployment:

1. Do not deploy with default `AUTH_SECRET`; fail startup if missing in production.
2. Protect `/api/chat` and all AI-cost endpoints.
3. Add rate limiting for login/signup/chat/extraction/PDF/validation.
4. Fix password change and use only `password_hash`.
5. Use secure cookies or a hardened token strategy.
6. Validate all backend payloads with Pydantic/JSON Schema before PDF generation.
7. Add upload/input size limits everywhere, not only email extraction.
8. Ensure LaTeX escaping is consistent for all nested structures.

10 Code Quality Scorecard

Category	Score	Comment
Architecture	7/10	Clear prototype architecture, but backend should be modularized.
Backend reliability	7/10	Core flows work. Needs automated tests, rate limits, validation, production server.
Frontend quality	6/10	Build passes, UX is rich, but lint errors and type looseness need cleanup.
Security	5/10	Basic auth exists; production hardening is pending.
Database design	6.5/10	User/docs storage works; chat persistence and migrations/schema discipline are pending.
Deployment readiness	5.5/10	Docker assets exist; image/process/runtime need cleanup.
Test coverage	3/10	Smoke testing works manually, but formal automated test suite is missing.
Overall	6/10	Strong demo/UAT base, not yet production ready.

11 Recommended Work Plan

11.1 Phase 1: Stabilize auth and backend health (1–2 days)

- Fix `/api/me/change-password` password hash bug.
- Add `@require_auth` to `/api/chat`.
- Add production startup guard for `AUTH_SECRET`.
- Add basic rate limiting.
- Add backend tests for signup, login, me, change password, document CRUD, and protected route failures.

11.2 Phase 2: Connect chatbot to MongoDB (1–2 days)

- Add chat session/message collections and indexes.
- Update `/api/chat` to persist user and assistant messages.
- Update frontend to load chat history from backend instead of `localStorage`.
- Add delete/clear conversation support.

11.3 Phase 3: Frontend cleanup (1 day)

- Fix ESLint errors, especially `any` types and hook warnings.
- Replace placeholder terms/forgot password behavior or hide until implemented.
- Improve token expiry handling and redirect behavior.

11.4 Phase 4: Deployment hardening (1–2 days)

- Switch Docker entrypoint to Gunicorn.
- Remove MongoDB from app image if using docker-compose MongoDB service.
- Pin dependencies.
- Add cleanup job for old PDFs.
- Add CI workflow with lint/build/smoke tests.

11.5 Phase 5: Regression and UAT (1–2 days)

- Run PDF generation smoke tests for FX NDF, IRS, CDS, and Equity TRS.
- Run AI extraction on all 10 sample emails and compare expected doc types/critical fields.
- Validate generated PDFs using the validation endpoint for representative samples.
- Test Docker Compose from clean checkout.

12 Final Recommendation

You have already completed the hard foundation: frontend, backend, DB connection, Gemini agents, and PDF generation. The remaining work is not a full rebuild; it is stabilization and productionization.

My recommendation: first fix auth and chatbot persistence, then clean frontend lint, then harden Docker/deployment. After these steps, the project can move from working prototype to deployment-ready MVP.

Go / No-Go: For demo or internal review: Go. For public/production deployment: No-Go until the blockers above are fixed.